TECHNICAL DOCUMENTATION

RisottOS

A compact 32-bit x86 educational operating system

Architecture, boot process, protected mode, Ring 3 execution, drivers, storage, and current engineering status.

Repository: risottos-team/RisottOs

Document version: 1.0

Prepared: 20 October 2026

Primary runtime: BIOS + QEMU i386

> **Scope note.** This document describes the repository as it exists today. Some subsystems are implemented as reusable kernel code but are not reached by the active boot path.

1. Executive overview

RisottOS is a freestanding, 32-bit x86 operating-system project intended for learning low-level boot, protected-mode execution, interrupt handling, device I/O, and the first steps toward user mode. It does not depend on an operating-system runtime or a standard C library once the kernel starts.

> **What happens today:** the BIOS boot sector loads the kernel, the kernel initializes the interrupt and timer infrastructure, prints its startup messages, creates a small machine-code program at a fixed address, and transfers execution to Ring 3. The test program invokes `int 0x80` once and then loops safely in user mode.

Area	Current status
Boot	BIOS boot sector, A20 enable attempt, EDD disk read, protected-mode entry.
Kernel	Freestanding 32-bit C kernel with assembly entry and interrupt stubs.
Protection	Flat GDT, IDT, TSS, Ring 3 transition, and a small syscall dispatcher.
Devices	VGA text/Mode 13h graphics, PIT, PS/2 keyboard, CMOS RTC, and ATA PIO.
Storage	Experimental RFS filesystem and raw ATA sector operations.
Not yet present	Paging, virtual memory, process isolation, robust exception handling, and complete userspace.

2. System architecture

The system is intentionally small and layered around direct hardware access.

BIOS → boot.asm → protected mode → kernel_entry.asm → kmain() → Ring 3 test app

Layer	Repository locations
Boot and CPU entry	`src/boot.asm` `src/kernel_entry.asm`

CPU architecture	`src/arch/idt.c`
	`src/arch/interrupts.asm`
	`src/arch/pic.c`

Kernel orchestration	`src/kernel.c`

Drivers	`src/drivers/` VGA, graphics, keyboard, timer, RTC, ATA
Core services	`src/mem/`, `src/rfs/`, `src/arch/syscall.c`
Interactive shell	`src/arch/shell.c` Implemented, but not called by the current `kmain()`.

3. Boot and kernel startup

1. **BIOS load:** the BIOS places the boot sector at `0x7C00`.
2. **Relocation:** the boot code copies itself to `0x0600`, protecting the original location while the kernel is loaded.
3. **A20 and disk:** it requests A20 enablement and uses BIOS EDD `INT 13h, AH=42h` to read 99 sectors from LBA 1 into physical address `0x1000`.
4. **Protected mode:** the bootloader loads an initial GDT, sets `CR0.PE`, performs a far jump, installs data segments, and sets the kernel stack to `0x90000`.
5. **Kernel entry:** `kernel_entry.asm` calls `kmain()`.
6. **Initialization:** the kernel configures the IDT, rebuilds the GDT and TSS, remaps the PIC, enables interrupts, starts the PIT and ATA support, and initializes a 4 MiB heap.
7. **User-mode test:** the kernel writes test instructions to `0x500000` and enters Ring 3 with `iretd`.

```
BIOS → relocate boot sector → load kernel at 0x1000 → protected mode
     → _start → kmain → IDT/GDT/TSS/PIC/PIT/ATA → Ring 3 smoke test
```

Hard-coded loader limit: the bootloader always reads 99 sectors and has no kernel-size metadata or bounds validation. The current kernel fits, but growth can silently make the image unsafe.

4. Memory map

Address / range	Purpose	Notes
`0x0600`	Relocated bootloader	Also uses the nearby real-mode stack.
`0x1000`	Kernel image	Flat binary loaded by the bootloader.
`0x8FFFF–0x90000`	Kernel/user transition stack area	`ESP0` and the initial user stack are hard-coded here.

`0xA0000`	VGA graphics framebuffer	Used by Mode 13h graphics routines.
`0xB8000`	VGA text memory	80×25 text output.
`0x100000–0x500000`	Kernel heap	4 MiB first-fit allocator arena.
`0x500000`	Ring 3 test application	Raw code and its test string are written here.

There is no paging or virtual-memory layer. The flat Ring 3 descriptors cover the full 4 GiB linear address space, so segmentation alone does not isolate user code from kernel memory or hardware mappings. This is a demonstration of privilege-level transition, not yet a secure userspace design.

5. GDT, IDT, PIC, and TSS

Global Descriptor Table

The bootloader initially defines six selectors: null (`0x00`), kernel code (`0x08`), kernel data (`0x10`), user code (`0x18`), user data (`0x20`), and TSS (`0x28`). The kernel later rebuilds an equivalent flat GDT in `src/arch/idt.c`, including the runtime address of its TSS.

Interrupt Descriptor Table

Vector	Source	Handler
`0x20`	PIT / IRQ0	Timer tick and EOI.
`0x21`	Keyboard / IRQ1	Scancode buffering.
`0x2E`	ATA / IRQ14	ATA completion status.
`0x80`	Software syscall	Ring 3-callable gate, attributes `0xEE`.

The PIC is remapped to vectors `0x20`–`0x2F`. The TSS supplies `SS0=0x10` and `ESP0=0x90000` when privilege transitions return to the kernel.

Exception handling gap: CPU exception gates are not installed. A divide error, invalid opcode, general protection fault, or page fault can therefore escalate into a triple fault and reset the virtual machine.

6. Ring 3 and syscall smoke test

`create_test_user_app()` constructs a tiny program directly in memory:

```
mov eax, 1              ; syscall number: print string
mov ebx, 0x00500064     ; pointer to message at app + 100
int 0x80
jmp $                   ; safe user-mode idle loop
```

`enter_usermode()` loads user data selectors and builds the five-word `iretd` frame: user SS, user ESP, EFLAGS with IF set, user CS, and user EIP. The CPU then begins execution at CPL 3.

Number	Operation	Arguments / result
1	Print string	`EBX` points to a NUL-terminated string.
2	Read keyboard	Returns a queued ASCII character or blocks while polling the keyboard controller.
3	Free memory	`EBX=pointer`.
4	Allocate memory	`EBX=size`; pointer returned in `EAX`.

The syscall handler saves registers, switches to kernel data segments, dispatches in C, restores state, and returns with `iretd`. The test application does not return to a scheduler or shell; it remains in its loop while timer interrupts continue to run.

7. Hardware and kernel subsystems

Subsystem	Implementation summary	Current boundary
VGA text	80×25 output, cursor, scrolling, colors, screen utilities.	Direct memory and port I/O.
VGA graphics	Mode 13h at 320×200 with 256-color palette, pixels, lines, shapes, and text.	Library code; not entered by the active boot path.
Keyboard	Set-1 scancode map, IRQ-driven 128-byte circular buffer, polling helper.	No shift, Caps Lock, extended keys, or modifier state.
PIT	Configurable channel 0 frequency and tick-based sleep.	Uses busy waiting with `hlt`.

RTC	Reads CMOS BCD hours, minutes, and seconds.	No time-setting or robust update-in-progress handling.
ATA PIO	Primary-master sector read/write, IRQ completion, flush, and IDENTIFY capacity.	Assumes a standard ATA-compatible primary IDE device.
Allocator	First-fit linked list, splitting, freeing, adjacent coalescing.	No alignment, validation, ownership, or concurrency protection.

8. RisottOS Filesystem (RFS)

RFS starts at absolute LBA 100 and stores fixed-size metadata on 512-byte sectors.

Relative area	Contents
0	512-byte superblock with magic `RFS1`, layout, and free-space data.
1 onward	Allocation bitmap.
After bitmap	16-sector table containing up to 128 file/directory entries.
After table	Contiguous file data sectors.

The implementation supports formatting, mounting, directory creation and deletion, file creation, replacement, reading, sizing, deletion, listing, and free-space reporting. Each entry is 64 bytes. The filesystem is non-reentrant, has no journaling, requires contiguous allocation, and is not mounted by the current `kmain()` path.

9. Shell and build system

Shell

`src/arch/shell.c` contains commands for screen clearing, echo, RTC access, sleep, ATA tests and raw sector access, graphics demos, memory operations, and RFS operations. It also protects low disk sectors from raw writes. The shell is currently unreachable because `kmain()` launches the Ring 3 smoke test instead of calling `start_shell()`.

Build

```
make          # assemble, compile, link, and create os.img
```

```
make run      # boot os.img in qemu-system-i386
make clean    # remove build/ and os.img
```

Tool	Role
`i686-elf-gcc`	Freestanding 32-bit C compiler.
`i686-elf-ld`	Links the flat kernel binary at `0x1000`.
`nasm`	Builds the boot sector and ELF32 assembly objects.
`qemu-system-i386`	Runs the raw 64 MiB disk image.

The Makefile collects C sources, links the kernel with `--oformat binary`, concatenates boot sector and kernel, and pads the result to 64 MiB. The checked-in `config/linker.ld` is not used by this Makefile. The legacy `scripts/build_and_run.sh` references a CMake configuration that is not present, so the Makefile is the authoritative build path.

10. Known risks and next steps

1. **Add exception handlers:** install diagnostic handlers for CPU faults before expanding user-mode code.
2. **Introduce paging:** create separate kernel and user mappings, guard the stack, and validate user pointers before syscall access.
3. **Replace fixed addresses:** derive kernel size, stack placement, heap bounds, and user image placement from a memory layout rather than hard-coded constants.
4. **Build a scheduler boundary:** define process state, user return policy, and a controlled path back from the test application.
5. **Harden interrupts and drivers:** validate ATA controller state and make interrupt-shared data access explicit. Keyboard IRQ acknowledgement now has one owner.
6. **Re-enable the shell intentionally:** mount or format RFS safely during startup and choose whether the shell or Ring 3 demo is the primary boot target.
7. **Improve RFS durability:** add allocation strategy, metadata ordering, recovery, and fragmentation handling.

> **Engineering perspective:** RisottOS has crossed the important boundary from a protected-mode kernel into a working privilege-level transition. The next milestone is not more Ring 3 instructions; it is isolation, fault reporting, and a controlled execution model around them.

Appendix A — Source map

Path	Responsibility
`src/boot.asm`	BIOS boot sector, disk load, initial GDT, protected-mode entry.
`src/kernel_entry.asm`	Assembly entry point calling `kmain`.
`src/kernel.c`	Startup order, heap initialization, Ring 3 test image.
`src/arch/idt.c`	IDT setup, GDT rebuild, TSS structure and descriptor.
`src/arch/interrupts.asm`	IRQ wrappers, syscall wrapper, TSS load, `iretd` transition.
`src/arch/syscall.c`	Syscall dispatcher for print, keyboard input, free, and allocate.
`src/arch/shell.c`	Interactive shell and command implementations.
`src/drivers/`	VGA, graphics, keyboard, timer, RTC, and ATA drivers.
`src/mem/`	Simple heap allocator.
`src/rfs/`	RFS metadata, directories, files, and allocation.
`Makefile`	Authoritative build, image creation, and QEMU run target.

This document is descriptive documentation, not a security or production-readiness certification.